

git add, git commit, git push, git add 会把当前修改加去 staging area，然后如果这个时候 commit，它会把 staging area（暂存区）里面的修改提交，但是没有加进 staging，直接 commit change 的时候，vscode 会自动把所以修改自动 git add 然后 commit，如果要自己在终端 add + commit，git commit -a -m “提交信息”（对 untracked 的文件，也就是新建的文件无效，需要先 git add.）。sync change（同步变更）会直接执行 git push 和 git pull，push 是把修改同步去远端仓库（github），pull 是把队友的修改同步到本机，先 pull 再 push，如果要在终端执行，git pull / push origin main。如果专案里面有全新建立的档案（Untracked，从未提交过的新档案），可以先用 git status 来确认状态，然后 git add.，这里的点代表全部底下的文件，也可以写 file name，如图所示。

```
C:\Users\Hoong\Desktop\git>git status
On branch dev
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        read.txt

nothing added to commit but untracked files present (use "git add" to track)
```

首先 git branch dev，创建一个 dev 分支，此时的 head 是指向 main，git checkout dev 来指向 dev 就会让 head 就会指向 dev，但是 origin/head 默认指向 remote repository 的 main 分支，之后可以通过远端仓库的 default settings 改去新分支，这样 origin/head 就是指向远端的 dev，前提是需要先 push 过去。此时这个分支只存在于本地电脑，如果要把他同步去远端仓库，就需要 git push -u origin dev（必须先执行 git checkout dev 来改变 head 指针），之后所有的 push 和 pull 都可以直接 git pull / push，不需要 git pull / push origin dev。概念图：

- 本地生孩子：git branch dev → 生出分支，但人還在 main。
- 靈魂切換：git checkout dev → 本地 HEAD 移情別戀，指向 dev。（這步是關鍵起手式！）
- 跨界相親：git push -u origin dev → 把腳下的 dev 送上 GitHub 建立遠端分支並綁定。
- 從此幸福：此後在 dev 底下，直接無腦 git push 或 git pull。
- 遠端大老：遠端的 origin/HEAD 預設看著 main，除非去 GitHub 後台改 Default Setting，它才會移情別戀到 origin/dev。

git checkout -b dev 等于执行：git branch dev（在當前位置建立名為 dev 的新分）+ git checkout dev（將 HEAD 指針切換到 dev 分支）

-b：代表 branch。这个参数会告诉 Git：如果这个分支不存在，请先帮我建立它

git push <远端主机名> <本地分支名>:<远端分支名>，把本机分支名推去远端主机名 origin，并命名为远端分支名，字面意思是让 origin 创建一个 dev 分支，然后 push 本机的 dev 去 origin，远端分支名没有写的话默认是本地分支名

在 Git 指令中，單個減號 - 後面只能接「單個字母」(例如 -d、-m)。如果後面接的是一個「完整的單字」(如 graph)，**必須使用兩個減號 --**

cd = change directory

git remote add origin <https://github.com/你的用戶名/仓库名.git>，意思是在遠端 add 一個 origin，就是我們的 url 地址

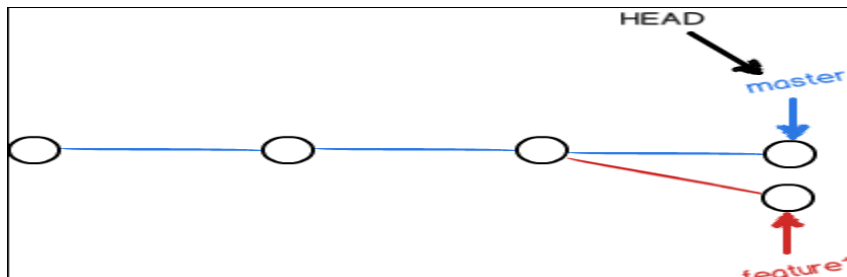
git init，把一個文件初始化成一個 git 的倉庫 (repository)，並在文件中自動添加一個 .git 隱藏文件，建議使用 git init -b main (git init + git branch -M main (把分支強制改成 main))，把主分支命名為 main，因為 github 的預設是 main，如果單單 git init，就會默認創建主分支名 master，

git restore，就是 discard 的意思，還沒 commit，不滿意改變？直接 discard，如果是 add 進了 staging area，里面的修改叫 staged change，就用 git restore --staged "file"，這個只是把他從 staged change 拉回來，代碼不會消失，單單 restore 會直接消失，但不会作用在 staged change 上面

git merge，先把目前的 HEAD 從 dev 切換回來 main，git checkout main，必須站在，main 上，才能把 dev 吸過來，所以下一步是 git merge dev，把本機的 dev 分支 merge 進 main 分支里面，然後 git push origin main，這個時候，遠端已經收到 merge 的信號，就能安心的 git branch -d dev 刪除 dev 本地分支了和 git push origin --delete dev 的遠端分支，最後 git branch 確認只剩下 main。如果你在 git push origin main 後面加一個: dev，如果我本來遠端沒有 dev，main: dev 就會把 main 推去遠端，並命名為 dev。如果已經有了 dev，就會把 main 覆蓋掉遠端的 dev，遠端的 main 保持不變，如果這個時候你的隊友在遠端的 dev commit 過文件，那就很幸運，因為系統會報錯，除非你執行 git push origin main: dev 前 pull 了 origin dev，或者沒有 pull 但是加個 -f 在 origin main: dev 前，這樣才會強行覆蓋

merge conflict (把图的 feature1 看成是 dev, master 看成 main)

首先 `git checkout -b dev`, 创建一个分支, 然后修改文件, `git commit -a -m "msg"`, 这个时候在切回 main 分支, `git checkout main`, 然后在 commit 一次修改, 这样 dev 和 main 各种有一次 commit, 变成了:



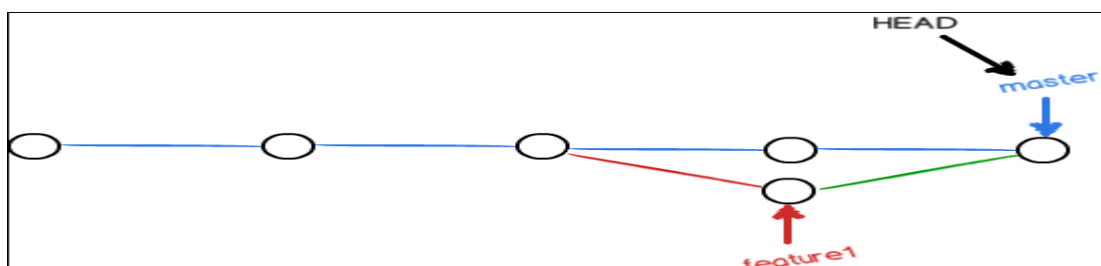
如果这个时候试图 `git merge dev`, 就会 error:

```
$ git merge feature1
Auto-merging readme.txt
CONFLICT (content): Merge conflict in readme.txt
Automatic merge failed; fix conflicts and then commit the result.
```

因为 main 和 dev commit 的是同一行代码, git 不知道要覆盖掉谁的, 就会把问题推会给我们解决, `git status` 就能看到 both modified, 意思是指本地的 main 改了这个档案, 要合并的 dev 也改了这个档案, 两边 (Both) 都动过它, 所以它现在处于「冲突待解决」状态, 这个时候打开 vsocde, 会看到一个文件

```
text
<<<<<<< HEAD
这是你在 master 分支写的内容 (比如: 高麗菜)
=====
这是别人在 feature1 分支写的内容 (比如: 大白菜)
>>>>>>> feature1
```

这个时候要么留 1 个, 要么 2 个都留, 或者可以把他们合并成同一行, 然后在 `git add` file 名 然后在 `git commit -m "fixed conflict"`, 不建议直接 `git commit -a -m ""`, 因为还处在 both modified 的状态, 最后先 add 进 staging area 再 commit, 这个时候就会变:



可以 `git branch -d dev`, `git log --graph` 看 graph

(git 分支的理解_转 - 你的雷哥 - 博客园)-----可参考

`git pull = git fetch + git merge`

`git fetch`, Git 会连接上 github, 把所有新 commit 下载到电脑里。

`git merge` 就会把下载的和你的代码合并, 如果这个时候你下载的和你的代码不一样, 那么就会 fast-forward (快速合并), 如果不小心 merge conflict 了, 提前恭喜你。所以正确做法是先 `git fetch` 下载, 然后 `git log HEAD..origin/main` (列出在 **origin/main** (远端最新) 有, 但在 **HEAD** (我本地目前) 没有的那些 Commit 纪录:

```
C:\Users\Hoong\Desktop\git>git log HEAD..origin/main
commit 2334979e525efa14d86d142625de8a4f9bb7d078 (origin/main, origin/HEAD)
Author: hoong1021 <hoong081021@gmail.com>
Date: Fri Jul 31 20:20:51 2026 +0800

    Update read.txt
```

Update read.txt 是远端最新的提交信息。应该先确认不会造成 merge conflict, 可以 `git diff HEAD..origin/main` 会把 2 边的代码一行一行对照出来。就可以 `git merge origin/main` 了 (把刚刚在第一步下载到 origin/main 抽屉里的内容, 手动拉过来与你目前的本地分支进行融合, 执行前必须 **checkout main**)。

cherry pick 是精准挑选某一个分支上某一次 commit, 把它复制到目前的分支上, 比如 main, 先 `checkout main`, 然后 `git cherry-pick <commit 的短 hash>`。如果要多个就 `git cherry-pick <a><b><c>`。

如果要直接复制 A-B 就 `git cherry-pick <起点短 hash..结束短 hash>`, 起点不会被复制, 终点会被复制。

当你 cherry pick 另一个分支的代码的时候, 本质上是把那个分支的代码复制一份到你目前指定的分支, 所以 2 个分支都有同样的代码内容, 只不过这个 hash 值不一样而已, 如果以后有人修改了任意一边的代码然后需要 merge 时, 会发生意外。

`git merge`, git 一上来不会先去看 2 个分支的代码有没有 conflict, 而是去看他们的 commit hash 一不一样, 如果一样, 会直接判定为 already up to date, 不需要再 merge, 因为 commit hash 一样代码肯定一样 (除非你去修改代码然后 commit, 这个时候才会有新的 commit hash 和新的 commit), 如果 2 边改的代码一个是第一行, 一个是第二行, 虽然 hash 值不一样, git 还是会完成 auto-merging, 或者说, main 分支改过代码, 但是 dev 分支动都没动过, 即使是同行, 也会是一个安全单方面修改, 因为 dev 根本没有动过

当执行 `git cherry-pick HashA HashB HashC` 批量摘樱桃, 结果在第二个 HashB 引发冲突时, 可以 `git cherry-pick --abort`, 会恢复到执行 cherry pick 的前一秒, 就算 hash A 没有问题, 也会被恢复。

git cherry-pick --continue, 先解决 hash b 的冲突, 然后 git add, git cherry-pick continue, git 就会继续执行后面的 cherry pick

stash 临时存储

在分支 dev 上写代码, 结果写到一半, 被要求切回 main 分支 fix bug, 如果这个时候直接 git checkout main, 就会:

```
C:\Users\Hoong\Desktop\git>git checkout main
error: Your local changes to the following files would be overwritten by checkout:
      read.txt
Please commit your changes or stash them before you switch branches.
Aborting
```

如果切换, 写到一半的代码可能会消失, 所以 git 要你先 commit 或者 stash, 这里不建议 commit 不完整的代码, 因为会弄脏这个 git 的 commit log (git log), 所以应该先 git stash, 意思是会把写到一半的代码存到 stash stack, 这个时候工作区(working directory)就变干净了, git status 也会显示 working tree clean, checkout main 就不会 error。如果解决完 main 分支的问题, 就可以 git checkout dev, 然后就可以 git stash pop 把不完整的代码拿出来继续写, 然后 git 也会帮你删掉留在 stash stack 的储存, 或者可以 git stash apply, 只是把它拿出来, 不会删除。如果怕解坏冲突可以利用 apply, 这样就能确保就算解坏了还有备份。

如果在一个新建的文件, untracked file, 这个时候你 stash 就没有屁用。可以 git stash -u, 告诉 git 这是一个 untracked file。也可以先 git add ., 然后随便 commit 一次先, 让它变 modified file。

stash pop conflict

git stash 的底層, 其實是 Git 在幕後偷偷幫你把這段半成品做成了「隱藏版的 Commit」鎖在祕密儲藏室裡。
當你執行 git stash pop 的時候, Git 的底層運作本質上就是一場 git merge (合併)! 它是把儲藏室裡的隱藏 Commit, 合併回你現在的分支!

1. 時空暫停前: 你在 dev 分支改了 readme.txt 的第一行, 寫著 x = 10 (半成品, 未 commit, 但被你 git stash 藏起來了)。
2. 時空穿梭修 Bug: 你切換回 main, 修 Bug 時不小心也改了 readme.txt 的第一行, 寫成了 x = 999, 並且成功 commit 存檔了。
3. 大合併發生: 你回到 dev, 把 main 合併進來 (或者你剛好人在 main 底下)。此時你的工作區裡, readme.txt 的第一行大喇喇寫著剛剛修好的 x = 999。